

Network Intrusion Detection System (NIDS)

Field: Cybersecurity **Difficulty:** Hard **Duration:** 8 weeks **Team size:** 1–2 interns

Context

Most companies rely on commercial SIEM/IDS products as black boxes. The goal of this project is to build, from scratch, a lightweight Network Intrusion Detection System that captures live traffic, extracts features, classifies flows as benign or malicious, and raises alerts in a small web dashboard. The intern will learn packet analysis, the anatomy of common attacks (port scan, SYN flood, brute-force, exfiltration), and applied ML on network data.

Objectives

- Capture packets on a network interface and reconstruct flows (5-tuple, bidirectional).
- Extract statistical features per flow (duration, byte counts, IAT, flag counts, etc.).
- Train a model to detect malicious flows using a public dataset.
- Run the model in (near) real-time on live traffic.
- Visualize alerts in a small dashboard with filtering and flow drill-down.

Mandatory part

1. Packet capture and flow reconstruction

- Live capture using `scapy`, `pyshark`, or `libpcap` bindings.
- Reconstruct flows by 5-tuple `(src_ip, dst_ip, src_port, dst_port, proto)`.
- Flow timeout (e.g. 120s idle, 600s active).
- Offline mode: replay `.pcap` files.

2. Feature extraction

At least 20 features per flow, including:

- Duration, total fwd/bwd packets, total fwd/bwd bytes.
- Mean/std/min/max packet size and inter-arrival time.
- TCP flag counts (SYN, ACK, FIN, RST, PSH, URG).
- Flow bytes/s, flow packets/s.

3. Detection model

- Train on **CICIDS2017** or **UNSW-NB15** (public datasets).
- Compare at least two models (e.g. Random Forest vs. XGBoost vs. a small MLP).
- Report precision, recall, F1 per attack class, confusion matrix.
- Persist the trained model and load it at runtime.

4. Real-time pipeline

- Capture → feature extraction → model inference → alert.
- Latency budget: < 5s between flow end and alert.
- Configurable threshold and ignore-list (IP/port).

5. Dashboard

- Web UI (any stack) showing:
 - Live alert feed (attack type, confidence, source/destination, timestamp).

- Flow table with filters.
 - Per-alert detail page (features, raw packets if available).
- Alerts persisted in SQLite or Postgres.

Bonus (only if mandatory part is complete)

- Detect a port scan / SYN flood with a rule-based engine in parallel with the ML model, and compare.
- Add a Suricata or Zeek integration to enrich alerts.
- Geolocation of source IPs.
- Slack/Discord webhook for high-confidence alerts.
- Live retraining loop with analyst feedback ("true positive" / "false positive" buttons).

Tech stack (suggested)

- **Capture / features:** Python + Scapy, or Go + gopacket.
- **ML:** scikit-learn, XGBoost, PyTorch (optional).
- **Storage:** SQLite (Postgres if scaling).
- **Backend:** FastAPI / Flask / Express.
- **Frontend:** React / Vue / Svelte.

Deliverables

- Git repo with clean history.
- `README.md` : install, run, dataset prep, demo screenshots.
- Trained model artifact + a `train.py` to reproduce it.
- A short report (5–10 pages) covering: dataset choice, feature engineering, model comparison, real-world limitations.
- Final live demo: run on a captured `.pcap` containing at least 3 attack types, show alerts on the dashboard.

Weekly plan

Week	Goal
1	Read about NIDS, attack categories, IDS evasion. Set up env. Capture/parse first packets.
2	Flow reconstruction + feature extraction. Replay <code>.pcap</code> files.
3	Dataset exploration (CICIDS2017). Baseline model.
4	Model comparison, hyperparameter tuning, metrics report.
5	Real-time pipeline: capture → features → inference → alert queue.
6	Dashboard MVP.
7	Hardening: threshold config, ignore-list, persistence. Bonus features.
8	Polish, write report, prepare final demo.

Evaluation grid

Criterion	Weight
Correctness of capture & flow reconstruction	15%
ML pipeline quality & metrics	25%
Real-time pipeline working end-to-end	20%
Dashboard usability	15%
Code quality, Git hygiene, tests	15%
Report & final demo	10%

Resources

- CICIDS2017 dataset: <https://www.unb.ca/cic/datasets/ids-2017.html>
- UNSW-NB15 dataset: <https://research.unsw.edu.au/projects/unsw-nb15-dataset>
- *Network Security Through Data Analysis* — Michael Collins.
- Suricata, Zeek, Snort documentation.

Prerequisites

- Solid Python (or Go).
- TCP/IP basics: handshake, flags, ports.
- ML basics: train/test split, overfitting, classification metrics.